



QairtGraph

Updated: Jul 02, 2026

80-63442-10

Rev: AL

Provide Feedback

 Note

Some methods in this module are not yet implemented in the current release and will raise an exception if called. See the C API for full functionality.

Include: `#include "QairtCppApi/QairtGraph.hpp"`

C++ wrapper for the QAIRT graph API.

namespace qairt

Enums

enum class Priority : std::underlying_type_t<[Qairt_Priority_t](#)>

Execution priority levels for graph scheduling.

Enumerator	Description
<code>Low</code>	Lowest scheduling priority.
<code>NormalLow</code>	Below-normal scheduling priority.
<code>Normal</code>	Normal scheduling priority.



Enumeration	Description
Default	Default scheduling priority, equivalent to Normal.
NormalHigh	Above-normal scheduling priority.
High	High scheduling priority.
HighPlus	Higher than High scheduling priority.
Critical	Critical scheduling priority.
CriticalPlus	Highest scheduling priority.
Undefined	Priority is not set or unrecognized.

Values:

enumerator Low = [QAIRT_PRIORITY_LOW](#)

enumerator NormalLow = [QAIRT_PRIORITY_NORMAL_LOW](#)

enumerator Normal = [QAIRT_PRIORITY_NORMAL](#)

enumerator Default = [QAIRT_PRIORITY_DEFAULT](#)

enumerator NormalHigh = [QAIRT_PRIORITY_NORMAL_HIGH](#)

enumerator High = [QAIRT_PRIORITY_HIGH](#)

enumerator HighPlus = [QAIRT_PRIORITY_HIGH_PLUS](#)

enumerator Critical = [QAIRT_PRIORITY_CRITICAL](#)



enumerator CriticalPlus = [QAIRT_PRIORITY_CRITICAL_PLUS](#)

enumerator Undefined = [QAIRT_PRIORITY_UNDEFINED](#)

enum class GraphError : std::underlying_type_t<[QairtGraph_Error_t](#)>

Error codes returned by QAIRT graph operations.

Enumerator	Description
MinError	Sentinel for the minimum error value.
NoError	Operation succeeded.
UnsupportedFeature	An optional API feature is not yet supported.
MemAlloc	Memory allocation failure in graph processing.
General	Unclassified graph error; any graph API may return this.
InvalidArguemnt	An argument to the graph API is invalid.
InvalidHandle	The provided graph handle is not valid.
GraphDoesNotExist	No graph with the specified name is registered in the backend.
InvalidName	Graph name is NULL, empty, or duplicates an existing name.
InvalidTensor	A tensor handle is NULL or invalid.
InvalidOpConfig	One or more elements of the op configuration are invalid.
SetProfile	Failed to bind the profile handle to the graph.
UnconnectedNode	A node was added before one or more of its input-producing nodes.
CreateFailed	Graph creation failed.

Provide Feedback



OptimizationFailed	Graph optimization failed with the specified ops or configuration.
FinalizeFailed	Graph finalization failed.
GraphNotFinalized	Attempted to execute a graph that has not been finalized.
GraphFinalized	Attempted to modify a graph after finalization.
ExecutionAsyncFifoFull	Async execution queue is full; no new requests can be registered.
SignalInUse	The supplied signal object is already in use by another call.
Aborted	Call aborted early due to a signal trigger.
ProfileInUse	The profile handle is already bound to another graph.
TimedOut	Call aborted early due to a signal timeout.
Subgraph	Operation is not permitted on a subgraph.
Disabled	The graph was disabled during context deserialization.
DynamicTensorShape	Dynamic tensor shape exceeded configured limits.
TensorSparsity	Tensor sparsity constraint violation.
EarlyTermination	Graph execution terminated early due to op-defined behavior.
InvalidContext	The context associated with this graph has already been freed.
MaxError	Sentinel for the maximum error value.
Undefined	Unused; present to ensure a 32-bit enum size.

Provide Feedback

Values:



enumerator MinError = QAIRT_GRAPH_MIN_ERROR

enumerator NoError = QAIRT_GRAPH_NO_ERROR

enumerator UnsupportedFeature = QAIRT_GRAPH_ERROR_UNSUPPORTED_FEATURE

enumerator MemAlloc = QAIRT_GRAPH_ERROR_MEM_ALLOC

enumerator General = QAIRT_GRAPH_ERROR_GENERAL

enumerator InvalidArguemnt = QAIRT_GRAPH_ERROR_INVALID_ARGUMENT

enumerator InvalidHandle = QAIRT_GRAPH_ERROR_INVALID_HANDLE

enumerator GraphDoesNotExist = QAIRT_GRAPH_ERROR_GRAPH_DOES_NOT_EXIST

enumerator InvalidName = QAIRT_GRAPH_ERROR_INVALID_NAME

enumerator InvalidTensor = QAIRT_GRAPH_ERROR_INVALID_TENSOR

enumerator InvalidOpConfig = QAIRT_GRAPH_ERROR_INVALID_OP_CONFIG

enumerator SetProfile = QAIRT_GRAPH_ERROR_SET_PROFILE

enumerator UnconnectedNode = QAIRT_GRAPH_ERROR_UNCONNECTED_NODE

enumerator CreateFailed = QAIRT_GRAPH_ERROR_CREATE_FAILED

enumerator OtimizationFailed = QAIRT_GRAPH_ERROR_OPTIMIZATION_FAILED

enumerator FinalizeFailed = QAIRT_GRAPH_ERROR_FINALIZE_FAILED

Provide Feedback



enumerator GraphNotFinalized = QAIRT_GRAPH_ERROR_GRAPH_NOT_FINALIZED

enumerator GraphFinalized = QAIRT_GRAPH_ERROR_GRAPH_FINALIZED

**enumerator ExecutionAsyncFifoFull =
QAIRT_GRAPH_ERROR_EXECUTION_ASYNC_FIFO_FULL**

enumerator SignalInUse = QAIRT_GRAPH_ERROR_SIGNAL_IN_USE

enumerator Aborted = QAIRT_GRAPH_ERROR_ABORTED

enumerator ProfileInUse = QAIRT_GRAPH_ERROR_PROFILE_IN_USE

enumerator TimedOut = QAIRT_GRAPH_ERROR_TIMED_OUT

enumerator Subgraph = QAIRT_GRAPH_ERROR_SUBGRAPH

enumerator Disabled = QAIRT_GRAPH_ERROR_DISABLED

enumerator DynamicTensorShape = QAIRT_GRAPH_ERROR_DYNAMIC_TENSOR_SHAPE

enumerator TensorSparsity = QAIRT_GRAPH_ERROR_TENSOR_SPARSITY

enumerator EarlyTermination = QAIRT_GRAPH_ERROR_EARLY_TERMINATION

enumerator InvalidContext = QAIRT_GRAPH_ERROR_INVALID_CONTEXT

enumerator MaxError = QAIRT_GRAPH_MAX_ERROR

enumerator Undefined = QAIRT_GRAPH_ERROR_UNDEFINED

Provide Feedback



```
enum class GraphProfilingState : std::underlying_type_t<QairtGraphProfilingState_t>
```

Profiling enabled/disabled state for a graph.

Enumerator	Description
Enabled	Profiling is active for this graph.
Disabled	Profiling is not active for this graph.
Undefined	Unused; present to ensure a 32-bit enum size.

Values:

```
enumerator Enabled = QAIRT_GRAPH_PROFILING_STATE_ENABLED
```

```
enumerator Disabled = QAIRT_GRAPH_PROFILING_STATE_DISABLED
```

```
enumerator Undefined = QAIRT_GRAPH_PROFILING_STATE_UNDEFINED
```

```
enum class TensorSetMemType
```

Values:

```
class Graph : public qairt::ApiType<Graph, QairtGraph_V1_t>
```

```
#include <QairtGraph.hpp>
```

Wrapper for a QAIRT graph handle.

Obtained via Context::createGraph() or Context::retrieveGraph().



Public Functions

```
~Graph() = default
```



```
Graph(const Graph&) = delete
```

```
Graph(Graph&&) noexcept = default
```

```
Graph &operator=(const Graph&) = delete
```

```
Graph &operator=(Graph&&) noexcept = default
```

Provide Feedback

```
inline void createGraphTensor(Tensor &tensor)
```

Create a tensor registered with this graph.

See also

QairtGraph_createGraphTensor

Parameters

tensor – [inout] Pre-configured tensor to register. The backend assigns a tensor ID directly to this handle as part of this call.

Throws

`qairt::Exception` – on:

- invalid graph or tensor handle
- invalid or unsupported tensor parameters
- memory allocation failure

```
inline void updateGraphTensors(const std::vector<Tensor*> &tensors)
```

Update previously created graph tensors with new data or quantization parameters.



- UPDATEABLE_STATIC tensors: data and quantization parameters.
- UPDATEABLE_NATIVE, UPDATEABLE_APP_READ, UPDATEABLE_APP_WRITE, UPDATEABLE_APP_READWRITE tensors: quantization parameters only.

See also

QairtGraph_updateGraphTensors

Parameters

tensors – [in] Array of pointers to tensors to update. Each tensor must carry the ID assigned during creation. Must not be empty.

Throws

`qairt::Exception` – on:

- invalid graph or tensor handle
- incompatible tensor update
- graph not finalized

Provide Feedback

```
inline void addNode(const OpConfig &opConfig)
```

Add an operation node to this graph.

Nodes must be added in dependency order: all native input tensors to the node must be outputs of a previously added node.

See also

QairtGraph_addNode

Parameters

opConfig – [in] Operation configuration describing the node to add. All tensors referenced must have been created via `createGraphTensor()`.

Throws

`qairt::Exception` – on:

- invalid graph handle



- graph already finalized
- node added out of dependency order

```
inline Graph createSubgraph(const std::string &graphName)
```

Create a named subgraph as a child of this graph.

A subgraph cannot be finalized or executed directly. Only a top-level graph with no parent can be finalized and executed. Nodes and tensors may be added to a subgraph before or after it is referenced in an op configuration.

See also

QairtGraph_createSubgraph

Parameters

graphName – [in] Unique name for the subgraph within the parent context. Must not be NULL or duplicate an existing graph name.

Throws

`qairt::Exception` – on:

- invalid or duplicate graph name
- invalid parent graph handle
- memory allocation failure

Returns

A new `Graph` object representing the created subgraph.

```
inline void setConfig(const GraphConfiguration &config)
```

Apply a configuration to this graph.

Modifies configuration options on an already-created graph. Must be called before `finalize`.
If the backend cannot support all provided configuration options, this call will fail.

QairtGraph_setConfig

Parameters

config – [in] Configuration object specifying priority, profiling, and custom options to apply.

Throws

`qairt::Exception` – on:

- invalid graph or configuration handle
- unsupported configuration option
- graph already finalized
- profile handle already in use by another graph

Provide Feedback

inline void finalize()

Finalize this graph for execution without a profiling handle.

Validates all operations, checks connectivity, and prepares the graph for execution. Some backends also require finalization of graphs retrieved from a context binary before execution.

See also

QairtGraph_finalize

Throws

`qairt::Exception` – on:

- invalid graph handle
- op or kernel creation failure
- graph optimization failure
- subgraph finalization attempt
- graph has zero nodes



```
inline void finalize(Profile &profile)
```

See also

[Graph::finalize\(\)](#)

```
inline void execute(IOTensorSet &ioTensors)
```

```
inline void execute(const std::vector<Tensor> &inputs, std::vector<Tensor> &outputs,  
std::shared_ptr<Profile> profile = nullptr, std::shared_ptr<Signal> signal = nullptr)
```

Execute this finalized graph synchronously with the given input and output tensors.

Blocks until execution completes. If other executions are already enqueued, this call waits in the same queue with equal priority to asynchronous calls.

See also

`QairtGraph_execute`

Parameters

- **inputs** – [in] Input tensors. Each must carry the ID assigned during [createGraphTensor\(\)](#). May be empty only if the graph has no application-writable tensors.
- **outputs** – [out] Output tensors to be populated by the backend. Each must carry the ID assigned during [createGraphTensor\(\)](#).
- **profile** – [in] Optional profile object for collecting execution metrics. Must be null if continuous profiling is configured via [GraphConfiguration::setProfile\(\)](#).
- **signal** – [in] Optional signal for aborting or timing out execution.

Throws

`qairt::Exception` – on:

- invalid graph handle
- graph not finalized
- subgraph execution attempted

- invalid or in-use signal
- set profile failed
- graph disabled during context deserialization
- dynamic tensor shape limit exceeded
- tensor sparsity constraint violated
- execution terminated early
- execution aborted or timed out
- context freed prior to execution

```
inline void execute(const std::vector<std::shared_ptr<Tensor>> &inputs,  
std::vector<std::shared_ptr<Tensor>> &outputs)
```

See also

[Graph::execute\(const std::vector<Tensor>&, std::vector<Tensor>&, std::shared_ptr<Profile>, std::shared_ptr<Signal>\)](#)

```
inline void executeAsync(const std::vector<std::shared_ptr<Tensor>> &inputs,  
std::vector<std::shared_ptr<Tensor>> &outputs, ApiTypeRef<const Profile&> profile,  
ApiTypeRef<const Signal&> signal, std::function<void(void*, NotifyStatus)> fn, void  
*notifyParam)
```

```
inline void executeAsync(const std::vector<std::shared_ptr<Tensor>> &inputs,  
std::vector<std::shared_ptr<Tensor>> &outputs, std::function<void(void*,  
NotifyStatus)> fn, void *notifyParam)
```

```
inline void executeAsync(const std::vector<std::shared_ptr<Tensor>> &inputs,  
std::vector<std::shared_ptr<Tensor>> &outputs, std::function<void(NotifyStatus)>
```



```
inline void executeAsync(const std::vector<std::shared_ptr<Tensor>> &inputs,
std::vector<std::shared_ptr<Tensor>> &outputs)
```

Private Functions

```
inline void customFree(handle_type handle)
```

```
inline Graph(const std::shared_ptr<ApiTable> &apiTable, QairtContext_Handle_t
contextHandle, const char *name, ApiTypeRef<const GraphConfiguration&>
graphConfig)
```

```
inline Graph(const std::shared_ptr<ApiTable> &apiTable, QairtContext_Handle_t
contextHandle, const char *name)
```

```
inline Graph(const std::shared_ptr<ApiTable> &apiTable, QairtGraph_Handle_t
parentHandle, const char *subgraphName)
```

Private Members

```
friend Context
```

```
bool m_isRetreived = false
```

True if this graph was retrieved from an existing context rather than created.

Private Static Functions

Provide Feedback

```
static inline void asyncCallbackTrampoline(void *trampolineObject, Qairt_Status_t status)
```

Friends

```
friend class Api
```

```
friend class ::qairt::ApiType
```

```
struct GraphRetrieveContext
```

Public Members

```
QairtContext_Handle_t m_contextHandle
```

```
class IOTensorSet
```

Public Functions

```
inline IOTensorSet(std::vector<Tensor> inputs, std::vector<Tensor> outputs)
```

```
inline std::vector<Tensor> &getInputs()
```

```
inline const std::vector<Tensor> &getInputs() const
```

```
inline std::vector<Tensor> &getOutputs()
```

```
inline const std::vector<Tensor> &getOutputs() const
```

Provide Feedback



Private Members

friend Graph

struct ParentGraphHandle

Public Members

QairtGraph_Handle_t m_parentHandle

class GraphConfiguration : public qairt::ApiType<GraphConfiguration, QairtGraph_ConfigV1_t>

#include <QairtGraph.hpp>

Configuration object for graph creation and execution behavior.

Construct directly – `GraphConfiguration()` – and call setter methods to configure priority, profiling, and custom options before passing to Context::createGraph().

Public Functions

GraphConfiguration() noexcept = default

GraphConfiguration(GraphConfiguration&&) noexcept = default

GraphConfiguration &operator=(GraphConfiguration&&) noexcept = default

inline GraphConfiguration &setCustomConfig(const GraphCustomConfig &config) 丕

See also

QairtGraph_Config_setCustomConfigs

Parameters

config – [in] Single custom configuration entry.

Throws

`qairt::Exception` – on invalid handle or invalid argument.

Returns

Reference to this configuration object, enabling method chaining.

inline void setCustomConfigs(const `GraphCustomConfiguration` &config**)**

Set multiple backend-specific custom configuration entries on this graph configuration.

See also

QairtGraph_Config_setCustomConfigs

Parameters

config – [in] Collection of custom configuration entries.

Throws

`qairt::Exception` – on invalid handle or invalid argument.

inline `Priority` getPriority() const

Get the scheduling priority for this graph configuration.

See also

QairtGraph_Config_getPriority

Throws

`qairt::Exception` – on invalid handle.

Returns

```
inline void setPriority(Priority priority)
```

Set the scheduling priority for this graph configuration.

See also

QairtGraph_Config_setPriority

Parameters

priority – [in] Desired scheduling priority level.

Throws

`qairt::Exception` – on invalid handle or invalid argument.

Provide Feedback

```
inline Profile &getProfile()
```

Get the profile handle bound to this graph configuration.

See also

QairtGraph_Config_getProfileHandle

Throws

`qairt::Exception` – on invalid handle.

Returns

Reference to the bound `Profile` object.

```
inline const Profile &getProfile() const
```

Get the profile handle bound to this graph configuration.

See also

QairtGraph_Config_getProfileHandle

Throws



Returns

Const reference to the bound [Profile](#) object.

```
inline void setProfile(const Profile &profile)
```

Set the profile handle on this graph configuration.

See also

`QairtGraph_Config_setProfileHandle`

Parameters

`profile` – [in] [Profile](#) object to bind.

Throws

`qairt::Exception` – on invalid handle or if the profile is already in use.

```
inline GraphProfilingState getGraphProfilingState() const
```

Get the profiling state for this graph configuration.

See also

`QairtGraph_Config_getProfilingState`

Throws

`qairt::Exception` – on invalid handle.

Returns

The current profiling state.

```
inline void setGraphProfilingState(GraphProfilingState graphProfilingState)
```

Set the profiling state for this graph configuration.

See also

Parameters

graphProfilingState – [in] Desired profiling state.

Throws

`qairt::Exception` – on invalid handle or invalid argument.

`inline uint32_t getNumProfilingExecutions() const`

Get the number of profiling executions configured for this graph.

See also

`QairtGraph_Config_getNumProfilingExecutions`

Throws

`qairt::Exception` – on invalid handle.

Returns

Number of executions to profile.

`inline void setNumProfilingExecutions(uint32_t numProfilingExecutions)`

Set the number of executions to profile for this graph configuration.

See also

`QairtGraph_Config_setNumProfilingExecutions`

Parameters

numProfilingExecutions – [in] Number of executions to profile.

Throws

`qairt::Exception` – on invalid handle or invalid argument.

Private Functions

```
inline GraphConfiguration(const std::shared_ptr<ApiTable> &apiTable,  
QairtGraph_ConfigHandle_t handle)
```

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

```
inline explicit GraphConfiguration(const std::shared_ptr<ApiTable> &apiTable)
```

Provide Feedback

Private Members

```
detail::crossable<detail::non_owning<Profile>, &interface_type::getProfileHandle,  
&interface_type::setProfileHandle> m_profile
```

Profile handle bound to this graph configuration for continuous profiling.

Friends

```
friend class Api
```

```
class GraphCustomConfig : public qairt::CustomConfigType
```

```
#include <QairtGraph.hpp>
```

Abstract base class for a single backend-specific graph custom configuration entry.

Public Functions

```
virtual ~GraphCustomConfig() = default
```

```
virtual QairtGraph_CustomConfigHandle_t getCustomConfigHandle() const = 0
```



Protected Functions

```
GraphCustomConfig() = default
```

```
GraphCustomConfig(const GraphCustomConfig&) = default
```

```
GraphCustomConfig(GraphCustomConfig&&) noexcept = default
```

```
GraphCustomConfig &operator=(const GraphCustomConfig&) = default
```

```
GraphCustomConfig &operator=(GraphCustomConfig&&) noexcept = default
```

```
class GraphCustomConfiguration
```

```
#include <QairtGraph.hpp>
```

Abstract base class for a collection of backend-specific graph custom configuration entries.

Public Functions

```
virtual ~GraphCustomConfiguration() = default
```

```
virtual std::vector<QairtGraph_CustomConfigHandle_t> getCustomConfigs() const = 0
```

Protected Functions

Provide Feedback



GraphCustomConfiguration() = default

GraphCustomConfiguration(const [GraphCustomConfiguration](#)&) = default

GraphCustomConfiguration([GraphCustomConfiguration](#)&&) noexcept = default

[GraphCustomConfiguration](#) &operator=(const [GraphCustomConfiguration](#)&) = default

[GraphCustomConfiguration](#) &operator=([GraphCustomConfiguration](#)&&) noexcept = default

struct NotifyStatus

Public Members

[QairtGraph_Error_t](#) error

class TensorSet

Public Functions

TensorSet() = default

TensorSet([TensorSet](#)&&) noexcept = default

Provide Feedback



```
TensorSet(const TensorSet&) = delete
```

```
TensorSet &operator=(TensorSet&&) noexcept = default
```

```
TensorSet &operator=(const TensorSet&) = delete
```

```
inline std::vector<std::shared_ptr<Tensor>> &getInputs()
```

```
inline const std::vector<std::shared_ptr<Tensor>> &getInputs() const
```

```
inline void setInputs(std::vector<std::shared_ptr<Tensor>> inputs)
```

```
inline TensorSetMemType getMemType()
```

```
inline TensorSetMemType getMemType() const
```

```
inline void setMemType(TensorSetMemType memType)
```

```
inline std::vector<std::shared_ptr<Tensor>> &getOutputs()
```

```
inline const std::vector<std::shared_ptr<Tensor>> &getOutputs() const
```

Provide Feedback




```
inline void setOutputs(std::vector<std::shared_ptr<Tensor>>> outputs)
```

← Previous

Next →

QairtContext

QairtTensor

Provide Feedback

May contain U.S. and international export controlled information

Light Dark Auto

Qualcomm

Qualcomm relentlessly innovates to deliver intelligent computing everywhere, helping the world tackle some of its most important challenges. Our leading-edge AI, high performance, low-power computing, and unrivaled connectivity deliver proven solutions that transform major industries. At Qualcomm, we are engineering human progress.



Quick links

- Products
- Support
- Partners

Company info

- About us
- Careers
- Investors

Stay connected

Get the latest Qualcomm and industry information delivered to your inbox.





© Qualcomm Technologies, Inc. and/or its affiliated companies.

Snapdragon and Qualcomm branded products are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

Note: Certain services and materials may require you to accept additional terms and conditions before accessing or using those items.

References to “Qualcomm” may mean Qualcomm Incorporated, or subsidiaries or business units within the Qualcomm corporate structure, as applicable.

Qualcomm Incorporated includes our licensing business, QTL, and the vast majority of our patent portfolio. Qualcomm Technologies, Inc., a subsidiary of Qualcomm Incorporated, operates, along with its subsidiaries, substantially all of our engineering, research and development functions, and substantially all of our products and services businesses, including our QCT semiconductor business.

Materials that are as of a specific date, including but not limited to press releases, presentations, blog posts and webcasts, may have been superseded by subsequent events or disclosures.

Nothing in these materials is an offer to sell or license any of the services or materials referenced herein.